

Kleo and Badem Robotic Arms

I built this leader-follower arm system so that I can move Kleo(leader arm) by hand and have Badem(follower arm) copy its movements. Three AS5600 sensors read Kleo's joints, and a Raspberry Pi Pico turns those readings into commands for Badem's servos. This document records how my current code connects everything and moves the arms.

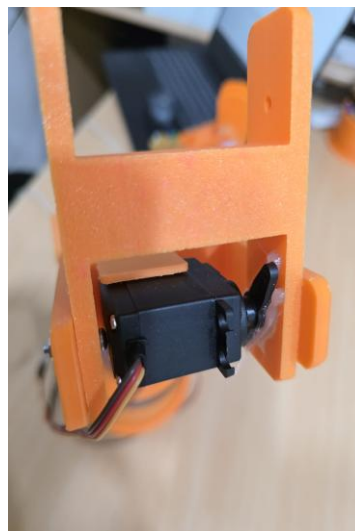
My first design



At first, I joined the arm pieces with screws and did not use a long shaft. The result looked a bit like blocks stacked on top of one another. It couldn't hold its own weight, and a lid-like piece kept slipping(can see the duck tape 😊). I tried printing the parts in several sizes, but the slipping continued. That was when I decided the structure needed to change.

How I built Badem - Follower

Badem uses three MG995 servo-driven joints built into 3D printed parts. The base joint turns the arm around the vertical axis, and the shoulder and elbow joints bend the links forward and backward. I mounted the servos in the printed structure, connected the moving sections with printed linkages, and used M3 screws for the fastened parts.

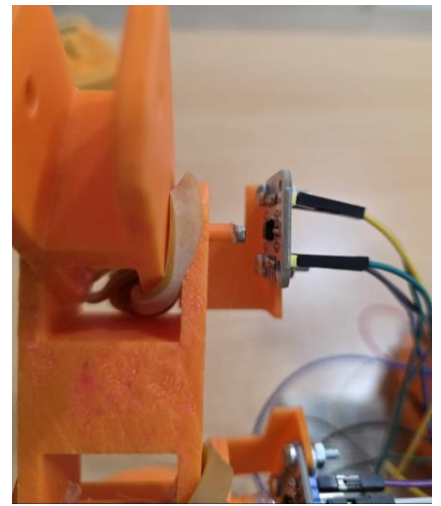
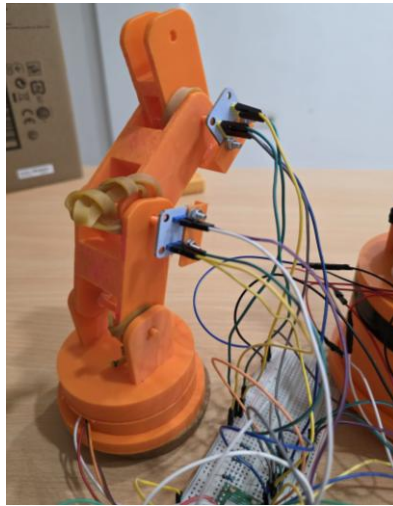


For the base, I designed two printed pieces that fit together like a can and its lid. I left enough clearance for the upper piece to turn, but kept the fit close enough that the pieces would not come loose as the arm moved.

I also used tape to secure a servo mount to the connected arm part. They were slipping apart instead of turning together when the servo moved. The tape kept that connection from sliding away, so the rotation could reach the arm part.

How I built Kleo - Leader

Kleo has one AS5600 at each of its three joints: base, shoulder, and elbow. At each joint, a magnet moves with the rotating shaft while the sensor sits beside it in a printed holder. From my tests and several tries, I got the best results between about 3 and 10 mm from the gap of the encoder chip to the magnet. Outside that range, I could not get stable readings. I fixed the magnets and encoder holders with silicone so their positions would stay consistent while I moved the arm.



Kleo's base uses the same can-and-lid idea as Badem's. The parts fit closely but still allow rotation.

Space around the sensor was tight, so I trimmed small pieces from the jumper wire connector heads to fit them inside. This kept the connectors from getting in the way of the encoder readings.

Why I used rubber bands

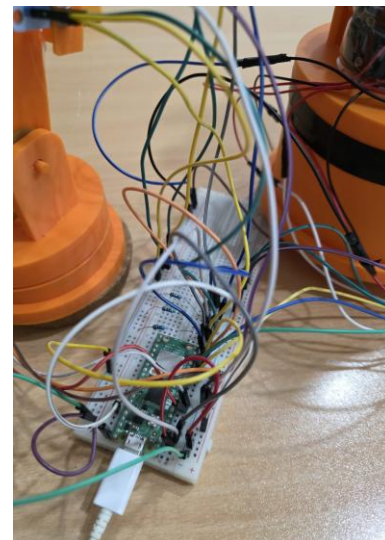
I fitted rubber bands around parts of Kleo's printed joint structure to increase friction where the pieces meet. Without them, those parts could slip instead of securing the position. The bands helped the pieces move together more reliably.

How the setup works

The three AS5600 sensors all use address 0x36, so I put each one on its own I2C bus. The base uses the Pico's hardware I2C1 bus, while the shoulder and elbow use SoftI2C. Each of Badem's servos has its own 50 Hz PWM signal. When I start the program, I place Kleo in its starting pose and press Enter to save that pose as the reference.

On the assembled arm, Badem's base turns around its vertical axis through 360°, while the shoulder and elbow bend forward and backward through approximately 180°.

Joint	AS5600 connection in code	Follower PWM pin	Commanded motion
Base	I2C1: SDA GP6, SCL GP7; 100 kHz	GP11	Circular 0–360° command
Shoulder	SoftI2C: SDA GP2, SCL GP3; 100 kHz	GP12	10–170° servo command
Elbow	SoftI2C: SDA GP14, SCL GP15; 50 kHz	GP13	10–170° servo command



I used 4.7 kΩ pull-up resistors on the shoulder and elbow I²C lines because of the AS5600 read errors.

The AS5600 problem

The encoders worked at first: the Pico could find them in an I²C scan. After I disconnected and reconnected one, it stopped giving usable readings. I kept the same wiring and checked the connections and voltages with a multimeter, but I couldn't find an obvious problem. I also tried reading its analog output on GP26 (ADC26). The values still jumped around even when I held the arm steady. At this point, the shoulder encoder was working normally.

I bought new AS5600 clones and ran into the same issue. My first attempt with pull-up resistors on SDA and SCL made the scan report every I²C address instead of just 0x36. Connecting DIR to ground didn't produce a reading on its own either. Eventually, the elbow worked after I used these changes together with power from the Pico's VBUS pin. Later, while I was recording a video, the shoulder developed the same issue. I changed its wiring too, and all three encoders then worked together. The base had worked without these extra steps, which made the difference especially confusing.

A note about my wiring: I honestly don't know which change made the elbow and shoulder work. My 4.7 kΩ pull-up resistors connect SDA and SCL to 5 V. Although the AS5600 supports 5 V power, those pull-ups can put 5 V on the Pico's 3.3 V GPIO pins; the resistors DO NOT make that safe. Please treat this as a record of what I tried, not wiring instructions to copy. Check your sensor module and use an I²C connection with compatible voltage levels.

Starting the arm

- First, the code checks that it can find an AS5600 at address 0x36 on each bus. If it cannot find one, it prints an error and waits instead of starting the servos.
- I place the leader arm in the pose I want to use as zero and press Enter. The code takes 30 readings from each sensor and averages them as angles, which also handles readings close to the 0°/360° boundary.
- The follower servos then receive their first commands: base 180°, shoulder 90°, and elbow 90°. After a one second pause, the arm begins following the leader.

How the follower moves

On each pass through the loop, the Pico reads a 12 bit angle from register 0x0C on every AS5600 and converts it to degrees. If a read fails, the code tries up to five times. It compares the new angle with the starting angle to find how far I moved each leader joint. The direction settings are -1 for the base, +1 for the shoulder, and -1 for the elbow; I can reverse a joint by changing its sign.

The code smooths each reading by mixing 20% of the new value with 80% of the previous filtered value. It ignores filtered movement smaller than 1°. The shoulder and elbow are kept within 10–170° commands. The base uses a circular 0–360° command and takes the shorter route between two commanded angles.

Control settings

Parameter	Base	Shoulder	Elbow
Center command	180°	90°	90°
Maximum change per loop	1.0°	0.7°	0.7°
Direction multiplier	-1	+1	-1
Pulse mapping	500–2500 μs over 0–360°	500–2500 μs over 0–180°	500–2500 μs over 0–180°

The programmed pause between updates is 20 ms; sensor reads and printing add extra time. The console shows the leader's relative angles beside the follower's current commands. If a read or another operation fails, the program prints the error, waits 100 ms, and tries again while the last PWM output stays active.

Using the current code

- Connect the three sensors and servo signal wires according to the table, with a common ground and suitable servo power.
- Run the program in a MicroPython serial console and check that all three scans find 0x36.

- Set the leader to its starting pose and press Enter. The follower receives its first servo commands at this moment.
- Move one leader joint at a time to check the matching follower joint and its direction.